



TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT NO. 1

Caesar Cipher over the Romanian Alphabet

C++ implementation with custom UTF-8 and UTF-32 conversion

Student: Gabriel Balan

Group: FAF-241

Coordinator: asist. univ. Zaică Maia

CONTENTS

Abstract	3
Introduction	4
1 Short theory	5
1.1 Standard Caesar cipher	5
1.2 Caesar cipher with a keyword permutation	5
2 Implementation	7
2.1 Representation and processing stages	7
2.2 Custom UTF-8 to UTF-32 decoder	7
2.2.1 Recognizing a sequence and reconstructing a code point	7
2.2.2 Structural and scalar-value validation	8
2.3 Custom UTF-32 to UTF-8 encoder	8
2.4 Romanian normalization and message validation	9
2.5 Numeric input and command-line control	10
2.6 Constructing and validating the permuted alphabet	10
2.7 Encryption, decryption and wraparound	11
3 Results	13
3.1 Standard and keyword-based encryption	13
3.2 Invalid arguments, characters and numeric keys	13
3.3 Supplementary verification	14
3.4 Scope relative to the assignment	15
Conclusions	16
Source code	17

ABSTRACT

This laboratory implements the standard Caesar cipher and its keyword-permuted variant for the 31-letter Romanian alphabet in C++. Beyond the substitution itself, the implementation includes a custom bidirectional UTF-8/UTF-32 conversion layer. It decodes variable-length byte sequences, checks their structure and scalar-value ranges, processes complete Unicode code points, and encodes results back into terminal-compatible UTF-8. Romanian case conversion and cedilla compatibility are implemented explicitly, while cipher arithmetic uses alphabet positions rather than Unicode values. The report explains these design decisions, presents the supplied terminal screenshots, and distinguishes the encryption interface from the decryption functionality available in the underlying functions. Supplementary checks verify both directions and the encoding boundaries without modifying the application source.

Keywords: Caesar cipher, C++, UTF-8, UTF-32, Romanian alphabet, bit masks, input validation

INTRODUCTION

The objective of Laboratory Work No. 1 is to implement and understand two Caesar cipher variants using the Romanian alphabet specified in the handout [1]. The numeric key belongs to $\{1, \dots, 30\}$; the second variant also uses a keyword of at least seven letters to reorder the alphabet. The exercise combines modular arithmetic, text normalization, validation, and an evaluation of the ciphers' security.

The implementation is contained in `Lab_1/src/main.cpp` [2]. Its CMake configuration requests C++23 and builds the executable `lab1` with strict compiler warnings. It uses the C++ standard library without an external Unicode library. The project extends the cipher exercise into a study of character encoding: instead of treating a terminal string as a sequence of single-byte letters, it explicitly converts between UTF-8 and UTF-32.

This separation matters for Romanian diacritics. A character such as `Ș` occupies two bytes in UTF-8 but must occupy one position in the cipher alphabet. The conversion functions also handle three- and four-byte sequences, allowing the application to distinguish valid Unicode outside the Romanian alphabet from malformed encoding. The report follows the actual source: the command-line argument selects one or two keys, while a Boolean function parameter controls decryption internally.

1 SHORT THEORY

1.1 Standard Caesar cipher

The Romanian alphabet has $n = 31$ letters in this fixed order:

AĂÂBCDEFGHIÎJKLMNOPSȘȚTUVWXYZ.

Its indices are $A = 0$, $Ă = 1$, $Â = 2$, $B = 3$, and so on through $Z = 30$. If x is the plaintext letter's index and y is the ciphertext letter's index, then

$$E_k(x) = (x + k) \bmod 31, \quad D_k(y) = (y + 31 - k) \bmod 31. \quad (1.1)$$

Adding 31 in decryption keeps the intermediate value nonnegative for an allowed key. The modulo operation wraps past the ends of the alphabet: with $k = 1$, Z encrypts to A , and A decrypts to Z .

The key space contains only 30 shifts, making exhaustive search practical. Uppercasing and deleting spaces are preprocessing operations; decryption recovers the normalized letter sequence, not the original capitalization or word boundaries.

1.2 Caesar cipher with a keyword permutation

The second key is a sequence of at least seven Romanian letters. Its first occurrences are written in order, followed by all unused letters of the natural alphabet. For the keyword **UCRAINA**, the repeated A is discarded during construction, producing

UCRAINĂÂBDEFGHÎJKLMOPQSȘȚTUVWXYZ.

The keyword has seven letters before deduplication and contributes six distinct letters. The length rule therefore does not require seven distinct letters.

If P denotes the resulting ordered alphabet and $\text{pos}_P(a)$ is the position of letter a in it, the transformations are

$$E_{k,P}(a) = P[(\text{pos}_P(a) + k) \bmod 31], \quad (1.2)$$

$$D_{k,P}(a) = P[(\text{pos}_P(a) + 31 - k) \bmod 31]. \quad (1.3)$$

Both lookup and output indexing use P . A lookup in the original alphabet followed by output from P would define a different transformation.

Allowing arbitrary keywords makes all $31!$ alphabet orderings attainable, so the nominal number of

alphabet–shift pairs is

$$30 \cdot 31! \approx 2.47 \cdot 10^{35}. \quad (1.4)$$

These pairs are not all distinct encryptions. Since 31 is prime, every nonzero shift visits all 31 positions before returning to its starting point. Reordering the alphabet relabels this cycle; consequently the distinct substitutions are the 30! possible cycles containing all 31 letters. This is still a monoalphabetic cipher: each occurrence of a given letter receives the same replacement. Letter frequencies and repeated-letter patterns remain available to an attacker, despite the much larger nominal key space.

2 IMPLEMENTATION

2.1 Representation and processing stages

The program uses three distinct notions of a character: UTF-8 bytes at the input/output boundary, Unicode code points in `std::u32string`, and indices from 0 to 30 for the cipher. The processing sequence is

UTF-8 input → UTF-32 decoding → normalization
→ alphabet validation → index-based substitution
→ UTF-8 encoding → terminal output.

Unicode values determine how a character is represented and recognized; they never determine its Caesar displacement. The alphabet constants provide the laboratory's required encoding directly:

```
constexpr std::u32string_view g_ralph = U"AĂÂBCDEFGHIÎJKLMNOPSŞŢŢUVWXYZ";  
constexpr std::u32string_view g_ralph_lower = U"aăâbcdefghiîjklmnopqrsşţţuvwxyz";  
constexpr long long emptySpace{U' '};
```

The `U` prefix creates UTF-32 literals. The two `constexpr std::u32string_view` objects refer to fixed uppercase and lowercase alphabets without owning a dynamically allocated copy. Their matching positions make Romanian case conversion independent of the terminal locale. `std::string_view` and `std::u32string_view` also let functions inspect existing strings without copying the input just to pass it as an argument; conversion outputs still own their storage.

2.2 Custom UTF-8 to UTF-32 decoder

2.2.1 Recognizing a sequence and reconstructing a code point

`utf8_to_utf32` advances a byte index through the input. The leading byte is converted to `unsigned char`, so high-bit byte values are processed as nonnegative values even on platforms where `plain char` is signed. Its leading bits identify the number of bytes:

Table 2.1 – UTF-8 sequence structure handled by the decoder

Bytes	Bit pattern	Scalar-value range
1	0xxxxxxx	U+0000 - -U+007F
2	110xxxxx 10xxxxxx	U+0080 - -U+07FF
3	1110xxxx 10xxxxxx 10xxxxxx	U+0800 - -U+FFFF*
4	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx	U+10000 - -U+10FFFF

*Excluding the surrogate range U+D800 - -U+DFFF.

For a two-byte sequence, `(b1 & 0xE0) == 0xC0` recognizes the `110` prefix. The expression

$(b2 \ \& \ 0xC0) == 0x80$ checks that the next byte starts with 10. After those checks, the payload bits are combined:

```
ch = static_cast<char32_t>(((b1 & 0x1F) << 6) | (b2 & 0x3F));
```

The first mask keeps five payload bits, shifting them left by six positions; the second keeps six bits. Bitwise OR joins them into an 11-bit value. For example, Ș is U+0218, encoded as bytes C8 98:

$$((C8 \ \& \ 1F) \ll 6) | (98 \ \& \ 3F) = (8 \cdot 64) + 24 = 536 = 0x0218. \quad (2.1)$$

The same idea extends to three bytes with shifts of 12 and 6, and four bytes with shifts of 18, 12 and 6:

```
ch = static_cast<char32_t> ( ((b1 & 0x07) << 18) |
                             ((b2 & 0x3F) << 12) |
                             ((b3 & 0x3F) << 6) |
                             (b4 & 0x3F) );
```

Each decoded value is appended once to the UTF-32 output. Romanian letters then occupy one string element each, and indexing cannot accidentally select half of a multibyte letter.

2.2.2 Structural and scalar-value validation

The decoder checks that enough bytes remain before reading a multibyte sequence, then validates every continuation-byte prefix. It also checks the reconstructed value against the range appropriate to that sequence length. These are separate checks: a sequence can have the correct prefixes while representing an invalid value.

The minimum values 0x80, 0x800, and 0x10000 reject overlong encodings, where a value is written using more bytes than necessary. The three-byte branch rejects surrogate code points, which are reserved for UTF-16 and are not Unicode scalar values. The four-byte branch rejects values greater than 0x10FFFF. Unexpected leading bytes, isolated continuation bytes, and incomplete sequences reach an error path. Structural failures terminate with status 1; the decoder’s reconstructed-value range failures terminate with status 2.

This creates a useful distinction. A valid four-byte emoji can be decoded correctly and subsequently rejected by the Romanian alphabet validator, with the whole character printed in the diagnostic. A malformed byte sequence cannot first become a valid internal character and causes immediate termination. The conversion layer understands a broader set of characters than the cipher permits.

2.3 Custom UTF-32 to UTF-8 encoder

utf32_to_utf8 performs the reverse operation. It selects one, two, three or four output bytes according to the scalar value, explicitly rejecting surrogate values and values above U+10FFFF. Payload

bits are extracted with right shifts and masks; fixed prefix bits are added with bitwise OR. The two-byte case is:

```
unsigned char b2{static_cast<unsigned char>((cur & 0x3F) | 0x80)};
unsigned char b1{static_cast<unsigned char>(((cur >> 6) & 0x1F) | 0xC0)};
output.push_back(static_cast<char>(b1));
output.push_back(static_cast<char>(b2));
```

The continuation byte receives its **10** prefix from **0x80**; the leading byte receives **110** from **0xC0**. Although the continuation byte is calculated first, the leading byte is appended first, preserving the required byte order. The three- and four-byte cases use the same construction with additional six-bit groups.

Encoding is necessary for more than the final ciphertext. When validation finds an unsupported code point, the program creates a one-element `std::u32string` and converts it back to UTF-8 for the error message. This is why the screenshot can identify a Hebrew letter or an emoji as a complete character instead of displaying an individual byte. The functions operate on code points; they do not segment user-perceived grapheme clusters or perform general Unicode normalization.

2.4 Romanian normalization and message validation

The `normalize` function first maps the historical cedilla variants to the modern comma-below forms, then converts lowercase letters using their position in the lowercase alphabet:

```
if (c == U'Ș')
    c = U'Ş';
else if (c == U'Ț')
    c = U'Ț';
else if (c == U'ș')
    c = U'ş';
else if (c == U'ț')
    c = U'ţ';

// Convert lowercase Romanian alphabet to uppercase
// using our alphabet ordering.

auto pos{g_ralph_lower.find(c)};

if (pos != std::u32string_view::npos)
    c = g_ralph[pos];
```

Thus Ș, Ş, ș, and ş become the same uppercase letter. Romanian diacritics remain distinct alphabet entries: Ă and Â do not become A, and Î does not become I. This preserves the specified 31-letter encoding.

The message validator decodes and normalizes the string, then searches for the first character that is neither a Romanian letter nor an ordinary space:

```
auto it = std::ranges::find_if(message, [](char32_t x) {
    return (g_ralph.find(x) == std::u32string_view::npos)
        && (x != emptySpace);
});
```

When such a character is found, it is encoded back to UTF-8 and reported; `getMessage` requests a new message. If validation succeeds, the caller's string is replaced with its normalized UTF-8 representation. Both cipher functions repeat normalization and alphabet checks internally, so direct calls do not depend on the interactive validator having run first.

Only `U+0020` is accepted as an internal message space and erased before transformation. The input expression `std::getline(std::cin >> std::ws, message)` consumes leading whitespace, including blank lines, before reading a line; it does not make internal tabs valid. No NFC composition is implemented: a precomposed `ă` is accepted, while `a` followed by a separate combining breve is rejected at the combining mark. Here “normalization” means the explicit Romanian mappings above.

2.5 Numeric input and command-line control

`getInt` reads an integer and marks the stream as failed if it is outside 1–30:

```
std::cout << "Enter a key value between 1 and 30: ";
int x{};
std::cin >> x;
if (x < 1 || x > 30)
    std::cin.setstate(std::ios::failbit);
```

The helper `clearFailedExtraction` handles extraction errors and leftover characters. It clears the stream state and discards the rest of the line before the next attempt. Consequently inputs such as `6abc` and `6.5` are rejected rather than silently becoming the key 6. The leftover-input check is strict: even a space after an otherwise valid number is rejected. Numeric end-of-input handling exits when the helper detects EOF.

The program is invoked as `./build/lab1 1` for the standard cipher or `./build/lab1 2` for the keyword variant. `main` checks the argument count before accessing `argv[1]`, and catches exceptions from `std::stoi`. The `begin` function then reads the message and shift, and uses a switch to select the variant; mode 2 additionally requests a keyword. The run ends after one operation.

Argument parsing is distinct from numeric-key validation: `std::stoi` is called without checking the number of consumed characters, so a prefix such as `1abc` can be interpreted as mode 1. Numeric modes other than 1 or 2 reach the switch's diagnostic after message and key input. The supplied multi-argument screenshot exercises the argument-count check specifically.

2.6 Constructing and validating the permuted alphabet

The keyword is decoded and normalized before its length is examined:

```
std::u32string keyword32{utf8_to_utf32(keyword)};
normalize(keyword32);

//keyword must be at least 7 characters
```

```

if (keyword32.size() < 7)
{
    std::cerr
        << "Keyword must contain at least 7 characters.\n";
    std::exit(1);
}

```

Because `keyword32.size()` counts code points, four two-byte Romanian letters cannot incorrectly satisfy a seven-letter requirement merely by occupying eight UTF-8 bytes. A subsequent loop rejects any character outside the Romanian alphabet, including an internal space. Invalid keywords print a diagnostic and terminate with status 1; there is no keyword retry loop.

The alphabet is constructed in two passes:

```

std::u32string permuted{};
permuted.reserve(g_ralph.size());

// First: distinct letters from keyword
for (char32_t c : keyword32)
{
    if (permuted.find(c) == std::u32string::npos)
        permuted.push_back(c);
}

// Then: remaining Romanian letters in natural order
for (char32_t c : g_ralph)
{
    if (permuted.find(c) == std::u32string::npos)
        permuted.push_back(c);
}

```

The first pass preserves the order of distinct keyword letters. The second supplies every missing Romanian letter in the required order. A letter is appended only if it is absent, so the completed alphabet contains all 31 letters exactly once. `reserve` allocates capacity for that known size without adding characters. The alphabet is printed after construction, making the second key's effect directly inspectable.

For `cryptographieeee`, the uppercase keyword contributes `CRYPTOGAHIE`. Repeated occurrences of *R*, *P*, and *E* do not create extra entries. The final alphabet is

CRYPTOGAHIEĂÂBDFÎJKLMNQSȘȚUVWXZ.

A longer keyword therefore need not produce a different alphabet or a stronger substitution; first-occurrence order is what determines the permutation.

2.7 Encryption, decryption and wraparound

Both cipher functions decode and normalize the message. They remove ordinary spaces with `std::erase` and reserve output capacity equal to the remaining number of code points. For each letter, `find` returns its position in the active alphabet. A failed lookup causes a diagnostic instead of allowing `npos` to enter the arithmetic. The standard version then applies:

```

std::size_t newPos{};

if (!decrypt)
{
    newPos =
        (pos + static_cast<std::size_t>(key))
        % g_ralph.size();
}
else
{
    // Adding 31 prevents unsigned underflow
    newPos =
        (pos + g_ralph.size()
         - static_cast<std::size_t>(key))
        % g_ralph.size();
}

result.push_back(g_ralph[newPos]);

```

The decryption branch adds the alphabet length before subtracting the key. This is particularly relevant because `pos` and `newPos` use the unsigned type `std::size_t`. With $1 \leq k \leq 30$, the intermediate value cannot underflow. The permuted version uses identical arithmetic with `permuted.find(c)` and `permuted.size()`.

Both functions expose `bool decrypt = false`; passing `true` selects the inverse transformation. However, `begin` calls them without this argument, so the current interactive interface performs encryption only. Numeric-key validation belongs to `getInt`; callers using the cipher functions directly must also respect the 1–30 precondition.

For a message of m code points and a keyword of q code points, alphabet searches inspect at most 31 entries. Transformation therefore takes $O(31m)$ time and permutation construction $O(31q + 31^2)$, both linear in input length for this fixed alphabet. The strings require linear storage. UTF-32 makes processing straightforward at the cost of four bytes per code unit; the application also performs some repeated conversions between its validation and cipher stages. The design favors explicit, inspectable steps over minimizing every allocation.

3 RESULTS

3.1 Standard and keyword-based encryption

Figure 3.1 is the supplied screenshot of two runs of the C++ executable. Both use the plaintext “Trimate două sute de mii de soldați să cucerească Moldova”. After uppercasing and removing spaces, the 48-letter message is:

TRIMITEDOUĂSUTEDEMIIDESOLDAȚISĂCUCEREASCĂMOLDOVA

For the standard alphabet and $k = 6$, the result is:

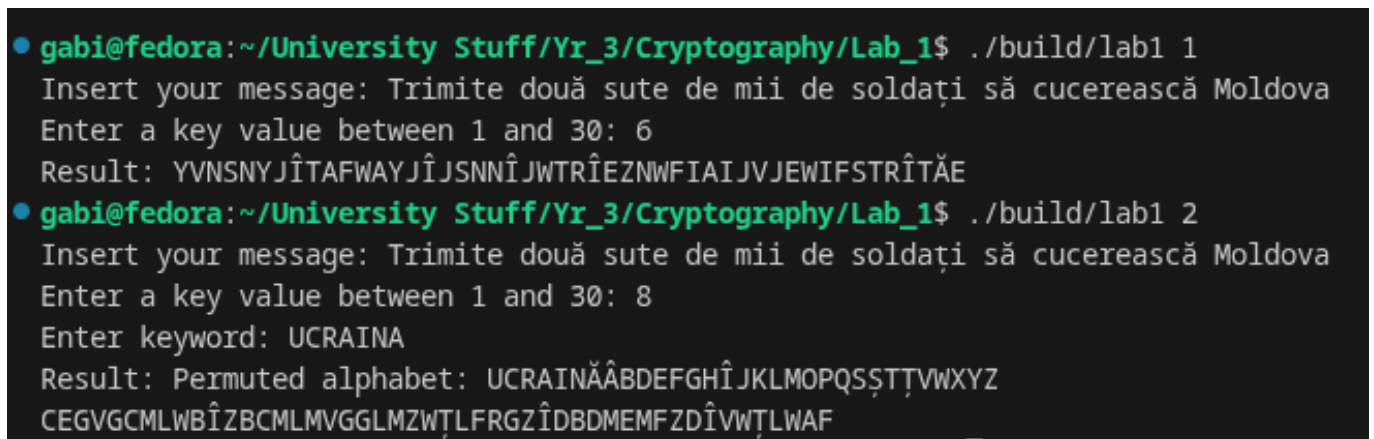
YVNSNYJÎTAFWAYJÎJSNNÎJWTRÎEZWNFIAIJVJEWIFSTRÎTĂE

For example, T has index 23 and becomes the letter at $(23 + 6) \bmod 31 = 29$, namely Y . Likewise, R at index 20 becomes V at index 26.

For $k = 8$ and keyword **UCRAINA**, the program prints the new alphabet and ciphertext:

UCRAINĂÂBDEFGHÎJKLMOPQSȘTȚVWXYZ
CEGVGCMLWBÎZBCMLMVGGLMZWTȚLFRGZÎDBDMEMFZDÎVWȚLWAF

In this alphabet T has index 24, so it maps to index $(24 + 8) \bmod 31 = 1$, which contains C . The changed ordering therefore explains why the two ciphertexts begin differently.



```
• gabi@fedora:~/University Stuff/Yr_3/Cryptography/Lab_1$ ./build/lab1 1
Insert your message: Trimate două sute de mii de soldați să cucerească Moldova
Enter a key value between 1 and 30: 6
Result: YVNSNYJÎTAFWAYJÎJSNNÎJWTRÎEZWNFIAIJVJEWIFSTRÎTĂE
• gabi@fedora:~/University Stuff/Yr_3/Cryptography/Lab_1$ ./build/lab1 2
Insert your message: Trimate două sute de mii de soldați să cucerească Moldova
Enter a key value between 1 and 30: 8
Enter keyword: UCRAINA
Result: Permuted alphabet: UCRAINĂÂBDEFGHÎJKLMOPQSȘTȚVWXYZ
CEGVGCMLWBÎZBCMLMVGGLMZWTȚLFRGZÎDBDMEMFZDÎVWȚLWAF
```

Figure 3.1 – C++ encryption with one key and with keyword **UCRAINA**

The literal `Result:` appears before `Permuted alphabet:` because the caller streams the result label before evaluating the cipher function, which itself prints the alphabet. The ciphertext is then returned and printed on the following line. This ordering is part of the existing console output.

3.2 Invalid arguments, characters and numeric keys

Figure 3.2 shows `./build/lab1 askjf 281937`. Two arguments were supplied where exactly one is expected, so the program prints `Usage: program.exe [nrKeys]` and returns status 1. The diagnostic retains the name `program.exe`; the actual CMake target on this system is `lab1`.

```

gabi@fedora:~/University Stuff/Yr_3/Cryptography/Lab_1$ ./build/lab1 askjf 281937
Usage: program.exe [nrKeys]

```

Figure 3.2 – Argument-count validation in the existing executable

Figure 3.3 demonstrates message retries after punctuation, an emoji, and Hebrew characters. The validator reports only the first unsupported code point in each message. Decoding succeeds for these well-formed Unicode characters; membership validation rejects them because the cipher accepts only the Romanian alphabet and ordinary spaces.

```

gabi@fedora:~/University Stuff/Yr_3/Cryptography/Lab_1$ ./build/lab1 2
Insert your message: aisdod .
. Is not a valid character!
Insert your message: ALKNFPO 😊
😊 Is not a valid character!
Insert your message: אהמ יעב
א Is not a valid character!
Insert your message: אה😊 saji
א Is not a valid character!
Insert your message: haah ok it works
Enter a key value between 1 and 30: -15
Oops, seems like you ARE INCOMPETENT ENOUGH TO WRITE A GOD DAMN NUMBER BETWEEN 1 AND 30. But it's ok, i forgive you. try again: Enter a key value between 1 and 30: 32
Oops, seems like you ARE INCOMPETENT ENOUGH TO WRITE A GOD DAMN NUMBER BETWEEN 1 AND 30. But it's ok, i forgive you. try again: Enter a key value between 1 and 30: 20
Enter keyword: cryptographieeee
Result: Permuted alphabet: CRYPTOAHIEĂÂBDFĲJLMNQŞŞTUVWVZ
WVWŢAXŞJŢNĂĂ

```

Figure 3.3 – Rejected characters and out-of-range shifts, followed by successful encryption

The accepted message is `haah ok it works`. The numeric inputs `-15` and `32` are rejected, and the program requests another key. The accepted key is $k = 20$. Using the keyword `cryptographieeee` then produces `WVWŢAXŞJŢNĂĂ`. Repeated letters make the substitution visible: the two *A* letters in the normalized plaintext both become *V*. The screenshots retain the program’s original diagnostic wording.

3.3 Supplementary verification

The CMake build completed using the configured C++23 standard and warning flags, including `-Werror`. The outputs above were reproduced from the compiled program and checked against the supplied screenshots.

To verify functionality that the console does not expose, a temporary C++ harness included the unchanged source with its entry point renamed by the preprocessor. It called both cipher functions directly. This check does not add a decryption menu or constitute an existing project test suite. The calls for the two example ciphertexts are equivalent to:

```

caesarCipher(standardCiphertext, 6, true);
caesarPermutationCipher(keywordCiphertext, 8, "UCRAINA", true);

```

Both returned the same normalized plaintext:

```

TRIMITEDOUĂSUTEDEMIIDESOLDAŢISĂCUCEREASCĂMOLDOVA

```

The harness also encrypted and decrypted all 31 alphabet letters for every allowed shift, once with the standard alphabet and once with the `UCRAINA` permutation. All 60 full-alphabet round trips passed. Additional

checks confirmed the screenshot ciphertexts, the exact keyword alphabet, repeated-keyword-letter handling, cedilla/case normalization, and wraparound.

Encoding checks round-tripped values at the one-, two-, three- and four-byte boundaries, across the scalar-value range. The tested values were U+0000, U+007F, U+0080, U+07FF, U+0800, U+D7FF, U+E000, U+FFFF, U+10000, and U+10FFFF. Mixed text containing Romanian letters, a euro sign, an emoji, and a Hebrew letter also round-tripped through the conversion functions. Eleven malformed UTF-8 inputs tested isolated continuation bytes, truncation, incorrect continuation prefixes, overlong encodings, a surrogate, an out-of-range value, and an invalid leading byte; all were rejected. Encoder checks rejected U+D800, U+DFFF, and U+110000.

Separate executable runs confirmed rejection of a six-letter keyword, the four-letter keyword `ăâîș` despite its eight UTF-8 bytes, and an internal keyword space. The short-keyword case can be reproduced from the `Lab_1` directory with:

```
printf 'ABC\n8\nROMANA\n' | ./build/lab1 2
```

It prints `Keyword must contain at least 7 characters.` and exits with status 1. These are supplementary execution checks, not additional screenshots supplied with the laboratory.

3.4 Scope relative to the assignment

The source implements both cipher directions, the required alphabet ordering, keyword permutation, custom Unicode conversion, and message/key validation. The submitted screenshots document encryption for both modes and several invalid-input cases. They do not show decryption or a short-keyword error. Decryption was verified at function level above; interactive operation selection remains absent, and invalid keywords terminate instead of being requested again. Character diagnostics identify the rejected character but do not list the entire permitted alphabet. These distinctions describe the current implementation without attributing features to it that belong only to the assignment specification.

CONCLUSIONS

The laboratory demonstrates that implementing a cipher over a non-ASCII alphabet involves two separate problems: representing characters correctly and transforming their positions correctly. The explicit 31-letter alphabet ensures that modular arithmetic follows the required Romanian encoding. The keyword variant changes that ordering while retaining a reversible substitution.

The custom UTF-8/UTF-32 layer was the most substantial extension of the exercise. Implementing it required identifying byte prefixes, extracting and assembling payload bits, checking sequence boundaries, rejecting overlong encodings and invalid scalar values, and restoring UTF-8 output. It enables code-point-based keyword length checks, predictable Romanian case handling, and diagnostics that preserve an unsupported multibyte character. Supporting all four UTF-8 sequence lengths also separates the general representation layer from the narrower alphabet policy.

The verified round trips confirm the inverse arithmetic for every permitted shift, while the console runs demonstrate normalization and recovery from invalid messages and numeric keys. The standard cipher remains vulnerable to a search over only 30 keys. The keyword variant enlarges the possible substitutions but retains letter frequencies and repeated-letter structure; neither construction provides modern confidentiality. The main learning outcomes are precise text representation, modular arithmetic, validation at multiple layers, and understanding the difference between encoding correctness and cryptographic security.

SOURCE CODE

The C++ implementation and CMake configuration are associated with the laboratory repository:

<https://github.com/AverageCEnj0yer/Cryptography>

The application is implemented in `Lab_1/src/main.cpp`. Its build configuration is stored in `Lab_1/CMakeLists.txt`. The report uses the existing UTM LaTeX class; its code excerpts are read directly from the application source at PDF build time.

BIBLIOGRAPHY

- [1] *Laboratory Work No. 1. Caesar Cipher*. Supplied course handout, Tasks 1.1 and 1.2 and report requirements, pp. 1–4.
- [2] Gabriel Balan. *Laboratory Work No. 1: C++ implementation*. Local source `Lab_1/src/main.cpp` and `Lab_1/CMakeLists.txt`, examined for this report.